

# md2D / md3D Practical Guide

Recipes for everyday studies: scans, convergence checks, lamellar and multilayer structures, rough surfaces, near-field maps and 3D structures, plus a troubleshooting table. The user manual defines every parameter used here.

- |                                            |                                        |
|--------------------------------------------|----------------------------------------|
| 1. Organising a study                      | 6. Near-field maps                     |
| 2. Scanning a parameter                    | 7. 3D structures and conical incidence |
| 3. Checking convergence                    | 8. Material indices                    |
| 4. Lamellar gratings and stacks            | 9. Troubleshooting                     |
| 5. Rough surfaces and non-periodic objects | 10. Older material in the repository   |

## 1 Organising a study

- Use one folder per study, holding the parameter file, the structure file and a script. Run the programs from that folder, since results are written to the current directory.
- Keep the parameter file for what does not change and give what varies on the command line. Build `profile_name` from the varying values: it names the result file, it may contain a sub-folder ( `-profile_name scan/theta_30` ), and a run overwrites an existing file of the same name.
- Start from a validated case in `validation_cases/` that resembles your structure. Its `param.txt` is known to work.

## 2 Scanning a parameter

One run per value, then one pass to collect the results. The example scans the angle of incidence for the aluminium grating of the manual's first run.

```
#!/bin/sh
# scan.sh - angle scan, results in scan/theta_<angle>.txt
MD2D=${MD2D:-$HOME/DifferentialMethod/md2D/md2D}
mkdir -p scan
for t in 0 10 20 30 40; do
    "$MD2D" -param tayeb_alu.txt -theta_i $t -verbosity 0 -profile_name scan/theta_$t
done
```

```
# collect.py - one line per angle
import sys; sys.path.insert(0, "/path/to/DifferentialMethod/validation_cases/tools")
from mdres import eff          # eff(file, "r"|"t", n) for md2D, eff(file, "r"|"t", nx, ny) for md3D
for t in (0, 10, 20, 30, 40):
    f = "scan/theta_%d.txt" % t
    print(t, eff(f, "r", 0), eff(f, "r", -1))
```

An order that is evanescent at some angle returns `None`. Wavelength scans work the same way with `-lambda`. If the indices change with wavelength, pass them too: `-n_sub_re ... -n_sub_im ...` (md2D) or `-nu_sub_re ... -nu_sub_im ...` (md3D). Independent runs can go in parallel ( `&` and `wait`, or GNU `parallel` ), one per core.

## 3 Checking convergence

Do this once per new kind of structure, on the most demanding configuration: the largest angle, the shortest wavelength and TM polarisation.

```
for NS in 250 500 1000 2000; do md2D -param p.txt -NS $NS -verbosity 0 -profile_name conv/NS$NS; done
for N in 20 30 40 60; do md2D -param p.txt -N $N -verbosity 0 -profile_name conv/N$N; done
grep -H one_minus_sum_eff conv/*.txt
```

- `one_minus_sum_eff` tracks the NS convergence for lossless structures, but **not** the N convergence. Compare the efficiencies you need between successive N.
- Typical outcome: dielectric gratings converge with slices of  $\lambda/100$ – $\lambda/1000$  and N just above the number of propagating orders. Metals in TM need slices of  $\lambda/1000$  or finer and N of 30–50.

## 4 Lamellar gratings and stacks

Binary structures (vertical walls, flat tops) are best described as a stack ( `MULTICOUCHES` ) and computed with `calcul_method = Z_INVAR` . Each z-invariant layer is then propagated exactly, like RCWA. Use enough slices that each stays thin,  $NS \geq N \cdot h/L$  : a single thick slice overflows at large N.

```
# line of material n1 (width 40 % of the period) on the substrate
utils/profilGen CARRE02_B -L1 0.4 -h1 0.2 -h2 0 -N_profil 1024 > line_raw.txt
{ echo "n1 = 1.46 + i0.0"; cat line_raw.txt; } > line.txt
```

- For more materials, add columns (one per interface, top to bottom) and indices `n2` , `n3` ... (manual § 5). `profilGen CARRE03` , `RECTANGLE04` and `TRAPEZ02` write such files.
- Sloped walls (trapezoids) are no longer z-invariant: use `RK4` with enough slices, or `Z_INVAR` as an explicit staircase approximation in TE only.
- `imposed_S_steps = AUTO` (md2D) places the slice boundaries on the horizontal interfaces found in the file, so that no slice straddles one.

## 5 Rough surfaces and non-periodic objects

A non-periodic surface is computed as one period of a large artificial period  $L$ . The diffracted orders then sample the scattering diagram with a step  $\Delta(\sin \theta) = \lambda/(vL)$ .

```
utils/profilGen ALEAT01 -N_profil 2048 -Ag 1 -Lg 20 -Ae 0 > rough.txt # Gaussian-correlated random profile
md2D -param rough_param.txt -L 50.3 -N AUTO -h 0.2 -profile_name rough_TE
```

The scattered intensity per unit angle (per radian) of each order is

$$dI/d\theta(\theta_n) = (L/\lambda) v \cos \theta_n e_n,$$

with  $v$  the index of the medium where it is observed.

- **N must cover every propagating order:**  $N \geq (\max(v_1, v_2) + v_1 |\sin \theta_i|) \cdot L/\lambda$ , plus a margin. `N = AUTO` does this. Too small an N silently drops orders while the energy balance still looks perfect. The cost grows as  $N^3$ , so an  $L$  of  $50\lambda$  means  $N \approx 110$  and about a minute per run in md2D.
- **Avoid Rayleigh anomalies.** If an order is exactly grazing ( $v \sin \theta_n = \pm v$  exactly, e.g.  $L = 50\lambda$  at  $\theta = 30^\circ$ ), the results are `nan` . Shift  $L$  or  $\theta$  slightly ( $L = 50.3\lambda$ ).
- The profile length  $L$  must be many correlation lengths long. The heights are rescaled to span `h` , so set `h` (peak-to-valley) from the wanted rms and the profile statistics. Average  $dI/d\theta$  over several random profiles for a smooth diagram.
- The same approach handles an isolated defect or line: put one object in a large period of flat surface.

## 6 Near-field maps

`calcul_type = NEAR_FIELD` (md2D) writes the field inside the computation box to the terminal, one row per slice (NS rows of  $N_x$  values). A complete example, a copper edge in silicon at 10.6  $\mu\text{m}$ , is in `Applications/ExempleUtilisationNearField/`.

```
md2D -param param_snomIR_edges.txt -calcul_type NEAR_FIELD -pola TM -verbosity 0 > field.txt
for c in ReEx ImEx ReEz ImEz ReHpy ImHpy; do utils/lire_tab2 $c < field.txt > $c.txt; done
```

- Put `h_min` / `h_max` in the structure file to extend the box above and below the structure, so the map includes some air and substrate.
- The vertical resolution is  $h/NS$  and the horizontal one  $L/N_x$ . Use `Z_INVAR` for binary structures: fields near sharp edges are singular and need a large  $N$  to resolve.

## 7 3D structures and conical incidence

### Biperiodic surface

Write an `H_XY` file from any function  $z(x,y)$ ; md3D computes the normals itself. For example, the pyramid of the validation cases:

```
python3 validation_cases/tools/mkprofile.py pyramid pyramid_512x512.txt --n 512
# or, for your own shape:
python3 - <<'EOF'
import math
n = 256
with open("mysurf.txt", "w") as f:
    f.write("profile_type = H_XY\nNpx = %d\nNpy = %d\nprofil =\n" % (n, n))
    for j in range(n):
        # rows: y
        f.write(" ".join("%.8e" % (math.sin(2*math.pi*i/n) + math.sin(2*math.pi*j/n))
                        for i in range(n)) + "\n")
        # columns: x
EOF
```

### Conical incidence on a 1D grating

Use md3D with `Ny = 0`, a one-row profile (`Npy = 1`) and any `Ly`.

`validation_cases/case06_conical_incidence_vs_gsolver/` is a complete example (lamellar grating,  $\theta = 30^\circ$ ,  $\varphi = 45^\circ$ ,  $\psi = 45^\circ$ ).

### Holes, pillars and other z-invariant patterns

Use an `N_XY_ZINVAR` index map with `calcul_method = Z_INVAR`. `NS = 1` is fine for thin patterned layers like this one ( $h = 0.05 L$ ); follow `NS  $\geq$  N·h/L` for thicker ones:

```
python3 validation_cases/tools/mkprofile.py circle holes.txt --n 1024 --R 0.25 --L 1.0 \
--n1 "1.0 + i0.0" --n2 "1.75 + i1.5"
```

Use 512 or more points per side; the 128-point map gives a  $3 \cdot 10^{-4}$  offset. The normal field is radial and centred on the cell, which suits centred round shapes only (manual § 8).

### Planning the cost

| <code>NX = NY</code> | <code>HARMONICS</code> | <code>TIME PER RUN (PYRAMID, NS = 20, OPENBLAS, AUTOMATIC THREADS)</code> |
|----------------------|------------------------|---------------------------------------------------------------------------|
| 4                    | 81                     | 1 s                                                                       |
| 6                    | 169                    | 4 s                                                                       |
| 8                    | 289                    | 26 s                                                                      |
| 10                   | 441                    | 1.6 min                                                                   |
| 15                   | 961                    | $\approx$ 20 min (extrapolated)                                           |

Time grows linearly with NS. With the reference BLAS instead of OpenBLAS these times are 3–8 times longer. Choose unequal `Nx` , `Ny` when the periods differ.

## 8 Material indices

- Indices go directly into the parameter or structure file as `n + ik` with  $k \geq 0$  for absorption. Tabulated data for most materials are at [refractiveindex.info](http://refractiveindex.info).
- For a wavelength scan, interpolate the table in your script and pass the index on the command line at each step.
- `utils/refractive_index` reads material files in a spectroscopic-ellipsometry format (see `md2D/Optimization_var_lambda/*_index.txt` ). Its lookup mode takes the wavelength in nm; the Cauchy mode is currently unreachable (the method argument is ignored).

## 9 Troubleshooting

| SYMPTOM                                                                            | CAUSE AND FIX                                                                                                                                                                                           |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ERROR</code> , can't read <code>"fft_filter"</code> (or another key)         | A required key is missing from the parameter file (manual § 4). Old parameter files lack <code>fft_filter</code> .                                                                                      |
| can't read <code>"l_smooth"</code>                                                 | <code>smoothing = 1</code> requires <code>l_smooth</code> (thickness of the transition layer). Older builds also showed this when <code>smoothing</code> was absent; the current code defaults it to 0. |
| <code>nan</code> efficiencies                                                      | An order is exactly grazing (Rayleigh anomaly). Change $L$ or $\theta$ slightly. With a metal substrate, <code>nan</code> in <code>theta_t</code> is normal.                                            |
| Efficiencies far above 1 or wildly varying (metal, TM)                             | Too few slices: increase <code>NS</code> until $h/NS \leq \lambda/1000$ .                                                                                                                               |
| Good energy balance but results change with N                                      | Not converged in N; energy conservation cannot detect it. Increase N.                                                                                                                                   |
| Weak orders off by a few %                                                         | Structure file too coarse for the number of harmonics. Use more points ( $\geq 2048$ in 2D for $N \approx 100$ ).                                                                                       |
| Orders appear mirrored ( $n \leftrightarrow -n$ ) compared with another code       | Different x or angle orientation. Mirror the profile or change the sign of $\theta$ .                                                                                                                   |
| <code>h determined ... is ...</code> and <code>h indicated ...</code>              | Information only: the profile is rescaled to <code>h</code> (x <code>coef_h</code> ). Use <code>h = AUTO</code> to keep the file's height.                                                              |
| A result file disappeared or was replaced                                          | Two runs used the same <code>profile_name</code> .                                                                                                                                                      |
| Debug lines ( <code>MxV</code> , <code>pointer par-&gt;...</code> ) at verbosity 0 | Leftover prints in md2D; harmless.                                                                                                                                                                      |
| md3D very slow                                                                     | Expected $\propto$ harmonics <sup>3</sup> . Check that OpenBLAS is in use ( <code>ldd md3D/md3D   grep blas</code> ), or lower <code>Nx</code> , <code>Ny</code> .                                      |

## 10 Older material in the repository

- `md2D/Optimization_var_lambda/` : fitting of a line's width and height to spectroscopic-ellipsometry data with Levenberg–Marquardt (scatterometry, thesis Part 2). It relies on a `calcul_type = ELLIPSO` that the current md2D no longer provides, so it needs porting before use.
- `Applications/` : the near-field example and two Octave scripts for post-processing ( `inverse_reflexion_TE.m` , `inverse_reflexion_TM.m` ).
- `PROVENANCE.md` : history of the code and of the removed older variants.